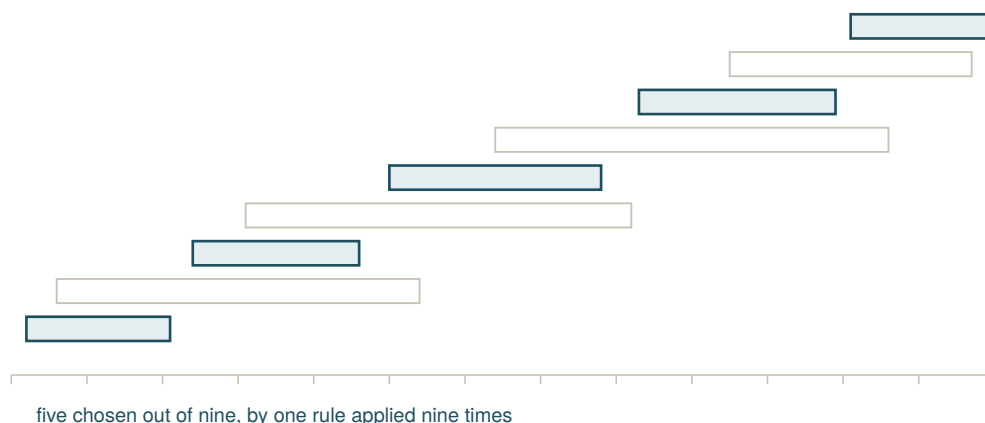


Greedy Algorithms

A beginner's workbook: pictures first, proofs second, code last.

Nine classic problems, each one taken apart the same way: draw it, guess a rule, try to break the rule, then prove it or throw it away.

Every algorithm here runs on a small example that you can follow by hand with a pencil. Do follow them by hand. Greedy algorithms are four lines of code and twenty minutes of thinking, and the thinking is the part that transfers to the next problem.



Assumed background: arrays, sorting, big-O notation.

Not assumed: dynamic programming, graphs.

Code: C++, but nothing beyond vector and sort.

Contents

1 What a greedy algorithm actually is 3

2 Proving a greedy rule: the exchange argument 5

3 Interval merging 6

4 Activity selection 9

5 Sweep line: counting what is active 12

6 Fractional knapsack 15

7 Job sequencing with deadlines 18

8 Jump Game: can you reach the end? 21

9 Jump Game II: the fewest jumps 23

10 Gas station 25

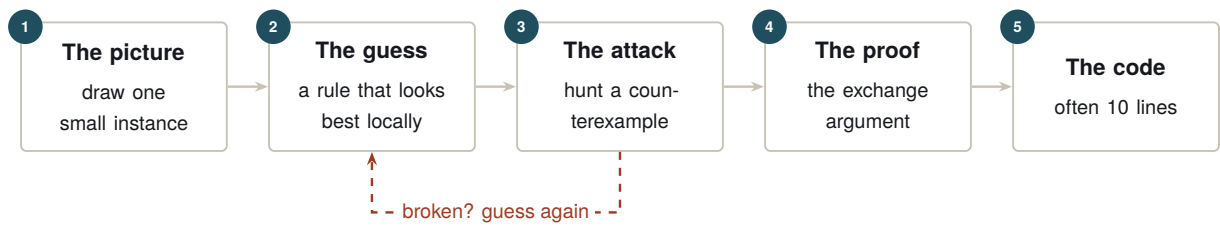
11 Choosing the rule: everything on one page 28

12 Exercises 30

13 Answers and hints 32

How to use these notes

Each problem chapter has the same five parts, in this order. Read them in order; the order is the point.



And four box styles recur throughout:

THE GREEDY CHOICE

The rule the algorithm follows. If you remember one line per chapter, remember this one.

WHY IT IS SAFE

The argument that the rule cannot lose. Skip these and you will write greedy code that fails.

CAREFUL

A rule that sounds equally reasonable but is wrong, with the input that kills it.

TRY THIS BEFORE READING ON

A question to answer on paper before reading on. Answers are in §13.

1 What a greedy algorithm actually is

You buy something for ₹ 62 and hand over ₹ 100. The cashier owes you ₹ 38 and does not sit down to enumerate every combination of notes. She takes the biggest note that still fits — 20 — then the biggest that still fits — 10 — then 5, then 2, then 1. Five notes, done in two seconds.

That is a greedy algorithm, and you have been running it your whole life.

THE SHAPE OF EVERY GREEDY ALGORITHM

Build the answer one piece at a time. At each step, look only at what is available *right now*, take the piece that looks best by some fixed rule, and never reconsider.

Three words in there carry all the weight.

- **Fixed rule.** “Largest note that fits” is a rule. It does not change based on what is left to pay.
- **Right now.** The rule never looks ahead. It does not simulate the rest of the process.
- **Never reconsider.** No backtracking, no undo. One pass down the decision tree.

That last point is the entire reason greedy is fast, and the entire reason it is dangerous.

1.1 One path instead of a tree

Any “choose a subset / choose an order” problem has a decision tree: at each node you pick one of the remaining options. Brute force walks the whole tree. Dynamic programming walks it too, but remembers repeated states. Greedy walks one root-to-leaf path and calls it the answer.

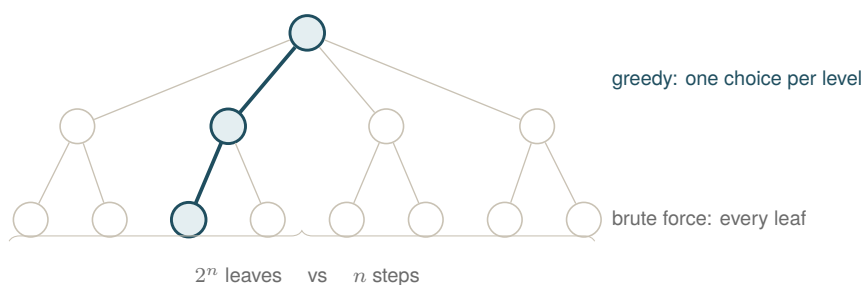


Figure 1. Greedy commits at every branch. That is a huge saving, and it is only correct if the branch it commits to always leads to an optimal leaf.

1.2 The rule that breaks

Indian currency happens to be *canonical*: for that set of denominations, grabbing the largest note is provably optimal. Change the denominations and the same rule falls apart.

TRY THIS BEFORE READING ON

Coins of value 1, 7 and 10, unlimited supply. Make 15 using as few coins as possible. Do it greedily first, then look for something better.

Greedy: take the biggest coin that fits**Optimal**

Figure 2. Greedy change-making on coins $\{1, 7, 10\}$ for a target of 15. The first choice, the 10, is the mistake — and greedy has no mechanism to take it back.

Notice what went wrong. The greedy rule was not stupid; 10 genuinely is the largest coin that fits. The problem is that taking 10 leaves a remainder of 5, which can only be paid in ones. The rule optimised the current step and wrecked every later step.

THE LESSON

A greedy rule is a *claim*, not a technique. “Take the best-looking option” is not an algorithm until you say what “best-looking” means and show that it cannot lose. Two different people will invent two different greedy rules for the same problem, and usually at most one of them is correct.

1.3 When is it worth trying?

Greedy has a real chance when both of these hold.

1. **Greedy-choice property.** There is some optimal solution that agrees with your first greedy pick. Taking it costs you nothing.
2. **Optimal substructure.** After making that pick, what remains is a smaller version of the same problem, and solving the remainder optimally gives an optimal whole.

Coin change on $\{1, 7, 10\}$ has property 2 but not property 1: no optimal solution for 15 contains a 10. That single failure is enough.

1.4 Greedy or DP?

Both eat problems with optimal substructure. The difference is how much they trust the first move.

	Greedy	Dynamic programming
Choices per step	one, committed	all, compared later
Needs a proof of	the choice itself	the recurrence
Typical time	$O(n \log n)$ (sorting)	$O(n \cdot W)$ or $O(n^2)$
Typical space	$O(1)$	$O(n)$ table or more
Failure mode	silently wrong answers	too slow / too much memory

A practical habit: when you suspect greedy, spend five minutes trying to break it on inputs of

size 3 or 4 before you write a line of code. Counterexamples to greedy are almost always tiny. If you cannot break it in five minutes, then go looking for the exchange argument.

2 Proving a greedy rule: the exchange argument

There is one proof technique that covers most greedy problems you will meet in a first course. It is worth learning properly once, because after that every chapter in this workbook is the same two paragraphs with different nouns.

2.1 The idea in one picture

You do not try to prove that your greedy solution *is* the optimal one. You prove something weaker and much easier: your first pick does not rule out optimality. Then you repeat.

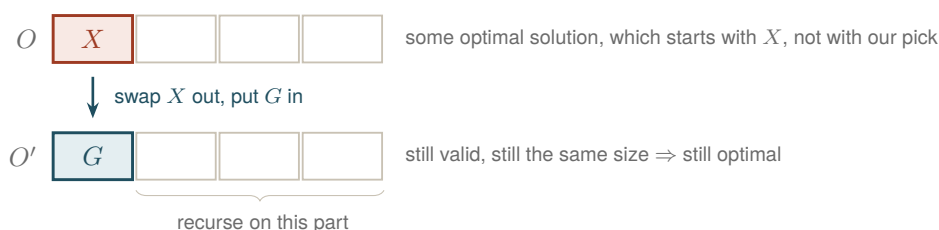


Figure 3. The exchange argument. G is the greedy pick, O is any optimal solution. If G can always replace whatever O picked first without damage, then some optimal solution begins with G .

2.2 The template

Write it out in this order every time. The only creative step is step 4.

- Step 1.** Name your greedy choice G precisely. “The activity with the smallest finish time” — not “the best activity”.
- Step 2.** Let O be *any* optimal solution. You know nothing about it except that it is valid and optimal.
- Step 3.** If O already contains G , there is nothing to do.
- Step 4.** Otherwise identify one element $X \in O$ that you can exchange for G , and show two things: the result is still **valid**, and it is **no worse**.
- Step 5.** Conclude: $O' = O - \{X\} + \{G\}$ is optimal and contains G . So committing to G loses nothing.
- Step 6.** The remaining problem is a smaller instance of the same problem. Apply the same argument to it.

TWO MISTAKES PEOPLE MAKE IN STEP 4

Proving the wrong direction. You do not need G to be *better* than X . You need it to be *not*

worse. Greedy solutions are usually tied with the optimum, not superior to it.

Assuming what O looks like. O is an arbitrary optimal solution. You cannot say “ O probably sorts its items the same way” or “assume O picks something reasonable”. If your argument needs O to be well behaved, it is not a proof.

2.3 A second technique: greedy stays ahead

Some problems are about a single running quantity rather than a set of picks — how far you can reach, how much fuel you have left, how many platforms are in use. For those, the exchange argument is awkward and induction is natural:

After k steps, greedy's value of the key quantity is at least as good as that of any other strategy after k steps.

If that holds at every k , greedy is never behind, so it cannot finish behind. Jump Game (§8) is the cleanest example: after processing index k , the greedy “farthest reachable index” is by construction the maximum over everything reachable, so no other strategy can be further along.

2.4 Three questions to ask before writing code

1. What am I sorting by?

If a greedy rule works, the input almost always needs one specific ordering. Name it.

2. What single state do I carry?

Last finish time, current balance, farthest reach, count of active intervals — usually just one number.

3. Can I break it at $n = 3$?

Spend five minutes. Counterexamples to wrong greedy rules are almost always tiny.

Everything from here on is an application of these three questions.

3 Interval merging

3.1 The problem

You are given a list of closed intervals, in no particular order. Overlapping ones must be fused into single intervals. Return the result.

input: [1,3] [8,10] [2,6] [9,12] output: [1,6] [8,12]

3.2 The picture

Draw them on a number line. The answer becomes obvious before you have thought about any algorithm — which is exactly why you draw them.

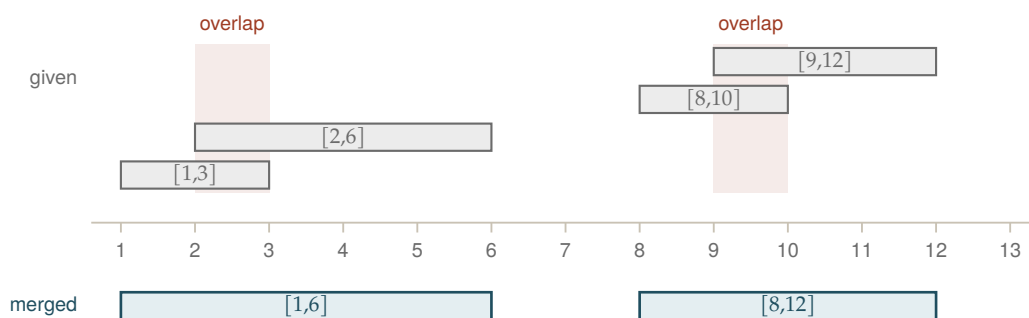


Figure 4. The two shaded columns are the only places where anything happens. Away from them the intervals are independent.

3.3 The guess

THE GREEDY CHOICE

Sort by **start** time. Keep the last interval of the answer open; extend its right end as long as the next interval starts inside it, otherwise close it and open a new one.

Why start time and not end time? Because sorting by start makes “does the next one touch what I have so far?” a single comparison. After sorting, if the next interval $[s, e]$ satisfies $s > r$, where r is the right end of the block being built, then no *later* interval can reach back to r either — their starts are all $\geq s > r$. The block is finished for good.

WHY IT IS SAFE

Let $B = [l, r]$ be the block currently open and let $[s, e]$ be the next interval in sorted order.

Case $s \leq r$. The two intervals share at least the point s , so any correct output must contain them in the same merged interval. Extending B to $[l, \max(r, e)]$ is forced. Note the max: the new interval may be entirely swallowed, as $[2, 4]$ is inside $[1, 9]$.

Case $s > r$. Every remaining interval starts at s or later, so nothing left can overlap B . Closing B now is safe — no later information can change it.

That second case is the whole reason sorting is not optional. Without a sorted order you have no guarantee that the interval which extends your block is the one you are about to look at.

TRY THIS BEFORE READING ON

Run the rule on $[1, 4]$ $[2, 3]$ and then on $[1, 4]$ $[5, 6]$. In the first case, what goes wrong if you write $r = e$ instead of $r = \max(r, e)$?

3.4 The trace

Sorted input: $[1, 3]$ $[2, 6]$ $[8, 10]$ $[9, 12]$.

Next interval	Test	Action	Answer so far
[1,3]	answer empty	open a block	[1,3]
[2,6]	$2 \leq 3$ overlap	$r = \max(3, 6) = 6$	[1,6]
[8,10]	$8 > 6$ no overlap	close, open new	[1,6] [8,10]
[9,12]	$9 \leq 10$ overlap	$r = \max(10, 12) = 12$	[1,6] [8,12]

3.5 The code

Merge overlapping intervals

```
vector<vector<int>> merge(vector<vector<int>>& a) {
    sort(a.begin(), a.end());           // by start, then by end
    vector<vector<int>> ans;

    for (auto& x : a) {
        if (ans.empty() || x[0] > ans.back()[1])
            ans.push_back(x);           // no overlap
        else
            ans.back()[1] = max(ans.back()[1], x[1]); // extend
    }
    return ans;
}
```

Sorting dominates: $O(n \log n)$ time, $O(n)$ extra space for the output ($O(1)$ if you are allowed to overwrite the input in place).

TOUCHING ENDPOINTS ARE A SPECIFICATION QUESTION, NOT A MATHS QUESTION

Are [1,3] and [3,5] one interval or two? Both answers are defensible:

- As *sets of real numbers* they share the point 3, so they merge into [1,5]. Use $x[0] > r$ to split, as above.
- As *half-open time slots* — a meeting from 1 to 3 followed by one from 3 to 5 — they do not conflict. Then use $x[0] \geq r$ to split.

One character of difference. The statement tells you which, and if it does not, the sample test does. Look before you submit.

3.6 The same trick, three more problems

Once the intervals are sorted by start, the single left-to-right scan solves a family of questions with almost no change:

Question	Change to the scan
Insert one new interval into an already-merged list	Same loop, with the new interval inserted at its sorted position.
Do any two intervals overlap?	Return <code>true</code> the first time $x[0] \leq r$.
Minimum intervals to erase so the rest are disjoint	On overlap, drop the one with the larger right end and keep the smaller — this is activity selection in disguise (§4).

4 Activity selection

This is the problem to understand properly. At least four later problems are activity selection wearing a different costume.

4.1 The problem

One lecture hall, n requests. Request i wants the hall from s_i to f_i . Two requests conflict if their intervals overlap. Accept as many requests as possible.

Note what is being maximised: the *count* of accepted requests. Not total hours used, not the importance of the requests. Just the count.

Activity	A	B	C	D	E	F
Start	1	3	0	5	8	5
Finish	2	4	6	7	9	9

4.2 The picture

$A \rightarrow B \rightarrow D \rightarrow E$: 4 activities, and no selection of 5 exists

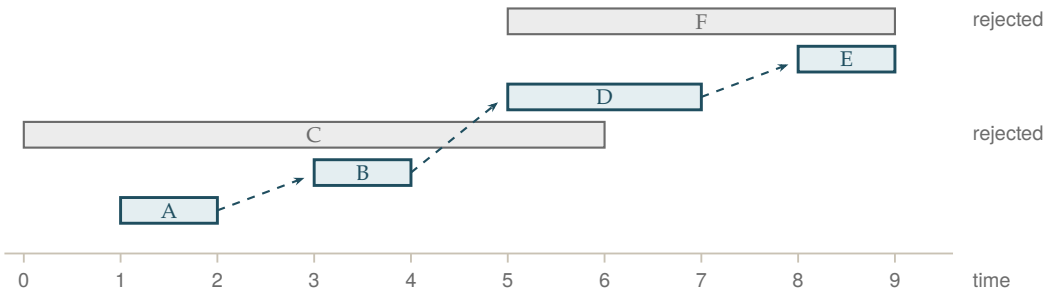


Figure 5. Rows are drawn in increasing order of finish time — that ordering is the algorithm. Each dashed arrow says “the hall is free again, take the next one that can start”.

4.3 The guess

THE GREEDY CHOICE

Sort by **finish** time. Walk the list once, accepting an activity whenever its start is at or after the finish of the last accepted activity.

The intuition in one sentence: the activity that frees the hall earliest leaves the largest possible block of time for everything else, and since all activities count the same, a larger remaining block can never hurt you.

4.4 The attack

Three rules that sound equally sensible. All three are wrong, and each dies to a three-activity input.



Figure 6. Two guesses that die on three activities. A short activity sitting on a boundary can destroy two longer ones; an early-starting activity can be long enough to destroy everything.

The third wrong rule is “fewest conflicts first”. It survives longer than the other two and needs a bigger counterexample — that is precisely why it is dangerous. Plausibility is not evidence.

4.5 The proof

Now the exchange argument, filled in.

EARLIEST FINISH TIME IS SAFE

Let G be the activity with the smallest finish time in the whole set, and let O be any optimal selection. Sort O by finish time and call its first activity X .

If $X = G$ we are done. Otherwise, since G has the minimum finish time over everything, $f_G \leq f_X$. Build

$$O' = O - \{X\} + \{G\}.$$

O' is **valid**. The only activity in O that could clash with G is one starting before f_G . Every activity in O other than X starts at or after $f_X \geq f_G$, so none of them clashes with G .

O' is **no smaller**. We removed one activity and added one, so $|O'| = |O|$.

So O' is also optimal and it contains G . Committing to G therefore costs nothing. Delete G and every activity overlapping it; what is left is the same problem on a smaller set, and the argument repeats.

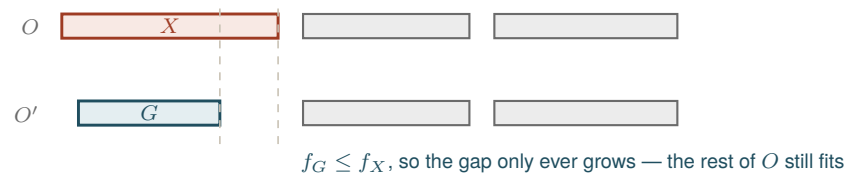


Figure 7. Swapping X for G pulls the first finish time *earlier*, which can only create room. Nothing downstream needs to move.

4.6 The trace

Sorted by finish time: A(1,2), B(3,4), C(0,6), D(5,7), E(8,9), F(5,9).

Activity	Start vs last finish	Decision	last_finish
A (1,2)	$1 \geq 0$	take	2
B (3,4)	$3 \geq 2$	take	4
C (0,6)	$0 < 4$	skip	4
D (5,7)	$5 \geq 4$	take	7
E (8,9)	$8 \geq 7$	take	9
F (5,9)	$5 < 9$	skip	9

Count = 4.

4.7 The code

Maximum number of non-overlapping activities

```
int maxActivities(vector<pair<int,int>> a) { // a[i] = {start, finish}
    sort(a.begin(), a.end(),
        [](const pair<int,int>& p, const pair<int,int>& q) {
            return p.second < q.second; // sort by FINISH time
        });

    int count = 0, lastFinish = INT_MIN;
    for (auto& [s, f] : a) {
        if (s ≥ lastFinish) { // hall is free
            count++;
            lastFinish = f;
        }
    }
    return count;
}
```

$O(n \log n)$ time, $O(1)$ extra space. If you also need the chosen activities, push them into a vector instead of counting.

THE ONE-CHARACTER BUG

`sort(a.begin(), a.end())` on a vector of `{start, finish}` pairs sorts by **start**, because that is

the first element. It compiles, it runs, and it is the wrong algorithm. Either store pairs as $\{\text{finish}, \text{start}\}$ or pass a comparator, as above. This is the single most common error in this problem.

4.8 Where it reappears

- **Non-overlapping intervals (minimum removals).** Keeping the maximum number is the same as removing the minimum. Answer: n minus the activity-selection count.
- **Maximum length of a chain of pairs.** A chain $(a, b) \rightarrow (c, d)$ needs $b < c$. Sort by second element; identical algorithm.
- **Minimum arrows to burst balloons.** Sort by right end, shoot at the right end of the first unburst balloon. Same proof, different wording.

5 Sweep line: counting what is active

The previous two problems asked *which* intervals to pick. This one asks *how many are on at once*. Different question, different technique — and the technique is worth more than the problem, because it reappears everywhere.

5.1 The idea

Forget intervals for a moment. Imagine a vertical line sweeping left to right across the number line. Nothing changes except at two kinds of moment: an interval begins, or an interval ends. So do not simulate time. Simulate **events**.

THE SWEEP

Turn each interval $[l, r]$ into two events: $+1$ at l and -1 at r . Sort all events by coordinate. Sweep through them keeping a running total. That total is the number of active intervals at every point, and its maximum is the peak overlap.

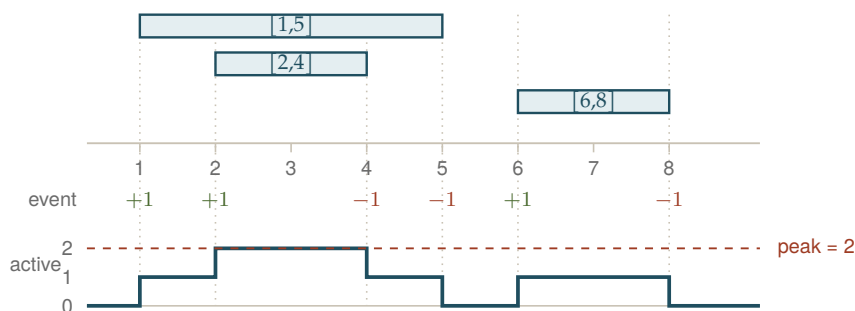


Figure 8. Three intervals, six events, one running count. The staircase answers every “how many at the same time” question; its height is what you report.

The sweep costs one sort plus one linear pass: $O(n \log n)$. There is nothing clever left in it, which

is the point — once a problem is phrased as events, it is finished.

5.2 Minimum platforms

The problem

Trains arrive and depart at a station. A train needs a platform from its arrival until its departure. What is the smallest number of platforms such that no train ever waits?

Train	T1	T2	T3	T4	T5	T6
Arrival	900	940	950	1100	1500	1800
Departure	910	1200	1120	1130	1900	2000

Reframing it

The answer is not about scheduling at all. A platform is occupied exactly while a train is present, so:

$$\text{minimum platforms} = \text{maximum number of trains present at the same instant.}$$

Which is the peak of a staircase. Sweep line, done.

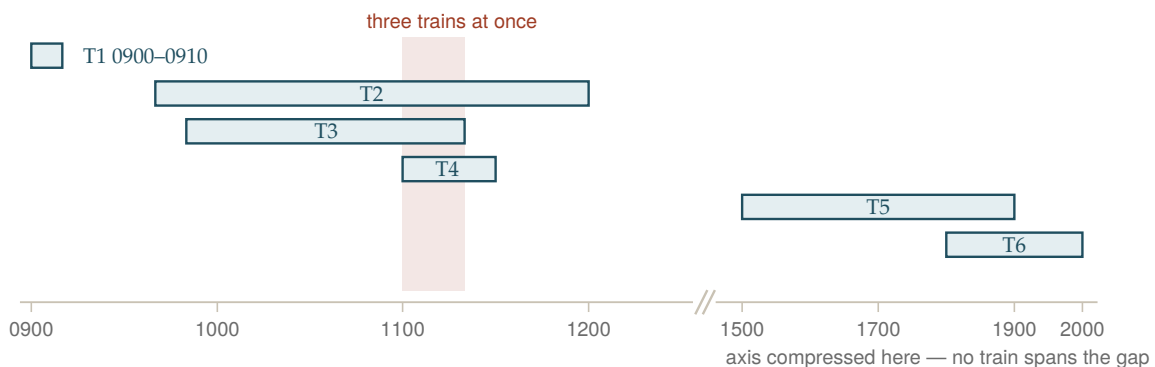


Figure 9. The trains drawn as intervals. Between 11:00 and 11:20 three of them are at the station at once, and nowhere do four overlap, so three platforms are both necessary and sufficient.

TRY THIS BEFORE READING ON

Before reading the code: without sorting anything, can you find the peak by checking, for each train, how many other trains overlap it? What is the running time of that, and why is it worth sorting instead?

Two pointers instead of an event list

You can build an explicit event array, but there is a tidier version. Sort the arrival array and the departure array *separately*, then merge-walk them. Sorting them independently destroys the pairing between a train's arrival and its departure — and that is fine, because the count at any instant does not care which train is which.

TWO-POINTER SWEEP

Repeatedly compare the next unprocessed arrival with the next unprocessed departure. Whichever is earlier happens next: an arrival means $cur++$, a departure means $cur--$. Track the maximum of cur .

The trace

Sorted arrivals: 900, 940, 950, 1100, 1500, 1800.

Sorted departures: 910, 1120, 1130, 1200, 1900, 2000.

a[i]	d[j]	Event	cur	max	Note
900	910	arrival	1	1	
940	910	departure	0	1	first train has left
940	1120	arrival	1	1	
950	1120	arrival	2	2	
1100	1120	arrival	3	3	peak
1500	1120	departure	2	3	
1500	1130	departure	1	3	
1500	1200	departure	0	3	station empty
1500	1900	arrival	1	3	
1800	1900	arrival	2	3	

Answer: **3 platforms**.

The code**Minimum platforms, two-pointer sweep**

```
int minPlatforms(vector<int> a, vector<int> d) {
    sort(a.begin(), a.end());
    sort(d.begin(), d.end());

    int n = a.size(), i = 0, j = 0, cur = 0, best = 0;

    while (i < n) {
        if (a[i] ≤ d[j]) {           // next event is an arrival
            cur++;
            best = max(best, cur);
            i++;
        } else {                   // next event is a departure
            cur--;
            j++;
        }
    }
    return best;
}
```

The loop only needs $i < n$: once every train has arrived, the remaining departures can only reduce cur , so they cannot change the answer. And j can never overtake i , so $d[j]$ is always a valid index.

THE BOUNDARY AGAIN

$a[i] \leq d[j]$ says: a train arriving at exactly the moment another departs needs its *own* platform. That is the usual reading — you cannot have two trains on one platform at the same instant, even briefly.

If the statement says a platform frees up immediately on departure and can be reused at the same minute, flip it to $a[i] < d[j]$. On the sample above the two readings give different answers, so the sample will tell you which one is wanted.

The same sweep, other names

Problem	What the staircase counts
Meeting rooms II	meetings in progress
Maximum people in a room	entries minus exits
Car pooling / bus capacity	passengers on board between stops
Maximum concurrent CPU jobs	jobs holding the CPU
Skyline / max building height	buildings covering an x coordinate

When the events are dense integers you can skip sorting entirely: bump a difference array, `diff[l]++` and `diff[r]--`, then take a prefix sum. That is $O(n + \text{range})$ and is often the fastest thing you can write.

6 Fractional knapsack

6.1 The problem

A bag holds W units of weight. Item i weighs w_i and is worth v_i . You may take any fraction of any item, and a fraction t of item i weighs $t w_i$ and is worth $t v_i$. Fill the bag to maximise total value.

The word *fractional* is doing an enormous amount of work here. Hold on to it.

Item	Value	Weight	Value per unit weight
A	60	10	6.0
B	100	20	5.0
C	120	30	4.0

Capacity $W = 50$

6.2 The guess

You are not really choosing items. You are choosing what to spend each unit of capacity on. So ask a different question: *if I have one spare kilogram, which item pays the most for it?*

THE GREEDY CHOICE

Sort by v_i/w_i , highest first. Take whole items while they fit. When the next item does not fit, take exactly the fraction that does, and stop.

6.3 The picture

Draw the bag as a bar of length W , and fill it from the left with the best-paying material first.

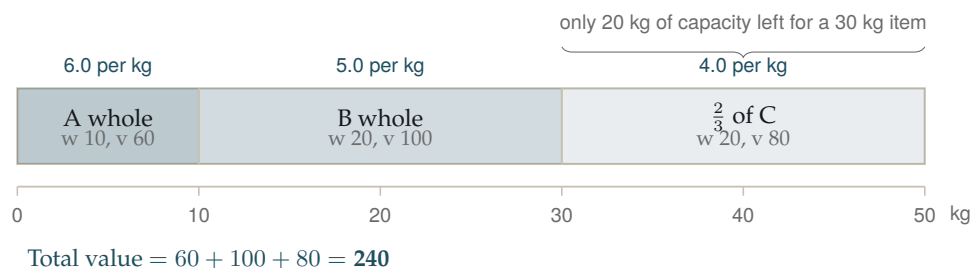


Figure 10. The bag filled in order of value per kilogram. The shading gets lighter as the material gets cheaper — the bar is never allowed to get darker as you move right, and that is the whole invariant.

6.4 Why it works**RATIO ORDER IS SAFE**

Think of the bag as W identical slots of one unit each. Every slot must be filled with material from some item, and a slot filled with item i yields exactly v_i/w_i .

Take any optimal packing. Suppose some slot holds material from item j while somewhere there is still unused material from item i with $v_i/w_i > v_j/w_j$. Swap one unit: put item i 's material in that slot and push item j 's material out. The weight is unchanged, so the packing is still valid, and the value changes by

$$\frac{v_i}{w_i} - \frac{v_j}{w_j} > 0,$$

so it improved — contradicting optimality.

Therefore in an optimal packing no slot is ever filled with cheaper material while dearer material remains available. That is exactly the greedy order.

Notice which assumption carried the argument: *one unit of material can be swapped for one unit of another*. That is only true because fractions are allowed. Remove fractions and the proof collapses — along with the algorithm.

6.5 The attack: the same rule on 0/1 knapsack**RATIO GREEDY IS WRONG WHEN ITEMS ARE INDIVISIBLE**

Capacity $W = 10$, and three items:

Item	Value	Weight	Ratio
A	12	6	2.0
B	9	5	1.8
C	9	5	1.8

Greedy takes A (best ratio, value 12), leaving 4 units of capacity, into which neither B nor C fits. Total: 12.

Taking B and C instead fills the bag exactly. Total: 18.

The 4 leftover units are worth nothing, and greedy had no way to see that coming. 0/1 knapsack needs dynamic programming; there is no repair to the greedy rule that fixes this.

TRY THIS BEFORE READING ON

Sorting by highest *value* instead of highest ratio is another tempting rule. Find a two-item input with capacity 10 on which it loses. (It does not take long — make one item heavy and barely worth it.)

6.6 The code

Fractional knapsack

```
struct Item { double value, weight; };

double fractionalKnapsack(vector<Item> a, double W) {
    sort(a.begin(), a.end(), [](const Item& x, const Item& y) {
        return x.value / x.weight > y.value / y.weight; // best ratio first
    });

    double total = 0.0;
    for (const Item& it : a) {
        if (W ≤ 0) break;
        if (it.weight ≤ W) { // whole item fits
            W -= it.weight;
            total += it.value;
        } else { // take the fraction that fits
            total += it.value * (W / it.weight);
            W = 0;
        }
    }
    return total;
}
```

$O(n \log n)$ for the sort, $O(n)$ for the pass, $O(1)$ extra space.

COMPARING RATIOS WITH DIVISION

$x.value / x.weight > y.value / y.weight$ is fine with doubles. With integers it truncates and silently produces a wrong order. Either cast to double, or — better, since it avoids floating point entirely — compare cross-products: $x.value * y.weight > y.value * x.weight$. Watch

for overflow if the values are large; long long is cheap insurance.

6.7 The distinction to memorise

	Fractional knapsack	0/1 knapsack
Can split an item	yes	no
Method	greedy on v/w	dynamic programming
Time	$O(n \log n)$	$O(nW)$
Leftover capacity	always zero (unless everything fits)	often wasted, and that is the difficulty

If an exam question gives you a knapsack, the first thing to find in the statement is whether items can be broken. That one word decides the entire solution.

7 Job sequencing with deadlines

7.1 The problem

Every job takes exactly one unit of time. Job i pays p_i , but only if it is finished by time d_i . One machine, so one job per time slot. Maximise total profit.

Job	A	B	C	D	E
Deadline	2	1	2	1	3
Profit	100	19	27	25	15

Because every job takes one unit, “finished by d_i ” just means “placed in one of the slots $1, 2, \dots, d_i$ ”. So the problem is really: assign jobs to numbered boxes, each box holding one job, job i only allowed in boxes up to d_i .

7.2 Two decisions, not one

Most greedy problems have a single rule. This one has two, and they do different jobs.

THE GREEDY CHOICE

Which job next? The most profitable one still unplaced.

Where to put it? The *latest* free slot at or before its deadline.

The first rule decides what you want. The second decides how not to get in your own way.

7.3 Why the latest free slot

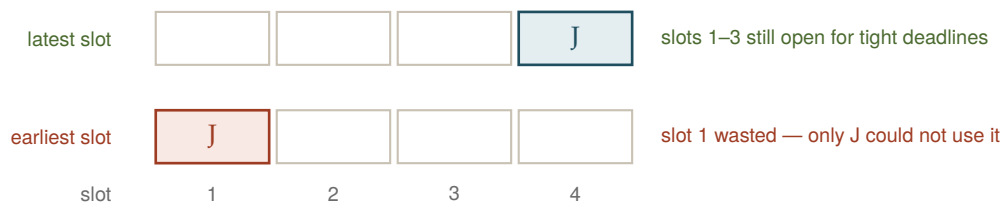


Figure 11. Job J has deadline 4. Both placements earn the same profit today. The top one leaves slot 1 free, and slot 1 is the only slot a deadline-1 job can ever use.

BOTH RULES TOGETHER

Highest profit first. Suppose an optimal schedule O leaves out the most profitable job J while including some job K . If O has a free slot at or before d_J , add J there — profit rises, contradiction. Otherwise every slot up to d_J is occupied; replace whichever of those jobs pays least with J . Since J pays at least as much as any job, profit does not fall, and the schedule is still legal because J sits at or before its own deadline. So some optimal schedule contains J .

Latest slot. Suppose J is placed at slot t while a later slot $t' \leq d_J$ is free. Moving J from t to t' changes no profit and breaks no deadline. The move can only help: slot t is usable by every job that could use t' , and by some that could not. So placing late is never worse than placing early.

Read that second paragraph again if the rule felt arbitrary. Late slots are the *least contested* resource, so spend them first and keep the contested early slots in reserve.

7.4 The trace

Sorted by profit: A(100, $d=2$), C(27, $d=2$), D(25, $d=1$), B(19, $d=1$), E(15, $d=3$). Maximum deadline is 3, so there are three slots.

Job	Slots tried (deadline down to 1)	Placed at	Schedule
A (100)	2 free	2	[_, A, _]
C (27)	2 taken, 1 free	1	[C, A, _]
D (25)	1 taken	—	rejected
B (19)	1 taken	—	rejected
E (15)	3 free	3	[C, A, E]

Profit = 27 + 100 + 15 = **142**, with three jobs done.

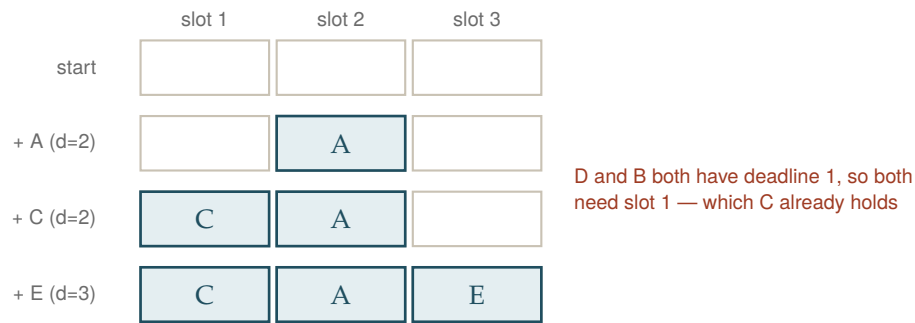


Figure 12. The schedule after each accepted job. Every job enters at the rightmost free slot it is allowed to occupy.

7.5 The code

Job sequencing with deadlines

```

struct Job { int deadline, profit; };

int jobSequencing(vector<Job> a) {
    sort(a.begin(), a.end(), [](const Job& x, const Job& y) {
        return x.profit > y.profit;           // most profitable first
    });

    int maxD = 0;
    for (const Job& j : a) maxD = max(maxD, j.deadline);

    vector<int> slot(maxD + 1, -1);           // slot[t] = job index, or -1
    int total = 0;

    for (int i = 0; i < (int)a.size(); i++) {
        for (int t = a[i].deadline; t ≥ 1; t--) { // latest free slot
            if (slot[t] == -1) {
                slot[t] = i;
                total += a[i].profit;
                break;
            }
        }
    }
    return total;
}

```

Sorting costs $O(n \log n)$. The inner scan costs at most D per job, where D is the largest deadline, so the whole thing is $O(n \log n + nD)$. For the input sizes in a first course that is completely fine.

7.6 Making the slot search fast

The only slow part is walking backwards looking for a free slot. If you union each filled slot t with $t - 1$ in a disjoint-set structure, then “latest free slot at or before d ” becomes a single `find(d)`, and the total drops to nearly $O(n \log n)$.



Figure 13. Disjoint-set shortcut. Grey slots are taken; each one forwards queries leftward, so a chain of full slots is skipped in one step.

Write the simple version first. Add this only when the constraints demand it.

CAREFUL

Two things routinely go wrong here.

Off-by-one in the slot array. Slots are numbered $1 \dots D$, so the array needs $D + 1$ entries and index 0 stays unused. Allocating D entries and indexing from 1 reads past the end.

Sorting by deadline. It feels natural and it is wrong: a job with an early deadline and tiny profit will evict a valuable one. Profit decides the order; the deadline only decides where a job is allowed to sit.

8 Jump Game: can you reach the end?

8.1 The problem

You start at index 0 of an array a . From index i you may jump forward to any index in $i+1 \dots i+a[i]$. Can you reach the last index?

$[2,3,1,1,4] \rightarrow \text{true}$ $[3,2,1,0,4] \rightarrow \text{false}$

8.2 The realisation

The obvious approach is to search: from each index, try every jump length. That is exponential, and dynamic programming over “is index i reachable” brings it down to $O(n^2)$. Both are more machinery than the problem needs.

Here is the observation that collapses it. You do not care *how* you got somewhere, or by which route. If index i is reachable at all, then from i you can reach anything up to $i + a[i]$. So the set of reachable indices is always a *prefix* — 0 up to some boundary — and one number describes it completely.

THE GREEDY CHOICE

Keep one variable, *far*, the largest index reachable so far. Walk left to right. At index i : if $i > \text{far}$ you are stranded, so return false; otherwise update $\text{far} = \max(\text{far}, i + a[i])$.

8.3 The picture

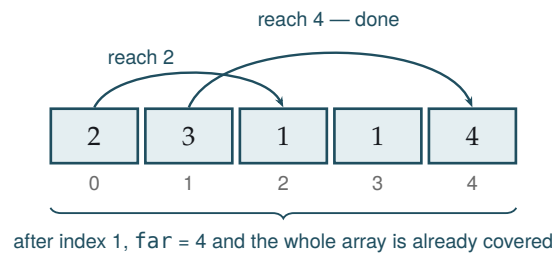


Figure 14. $[2, 3, 1, 1, 4]$. The boundary only ever moves right, and it is the only thing worth remembering.

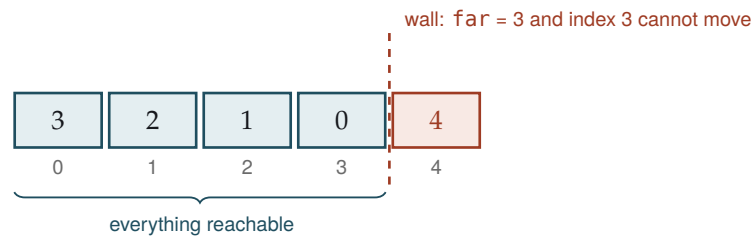


Figure 15. $[3, 2, 1, 0, 4]$. Whatever route you take, the boundary stalls at index 3. The zero is a wall, not a detour.

8.4 Why one number is enough

GREEDY STAYS AHEAD

Let R_k be the set of indices any strategy can reach using only positions $0 \dots k$, and let F_k be the value of far after the algorithm has processed index k .

Every index in R_k is at most $\max_{i \leq k} (i + a[i])$, because a jump from i lands no further than $i + a[i]$. And F_k is exactly that maximum, by construction. So $F_k \geq$ every reachable index, at every k .

Greedy is therefore never behind any strategy, so if the last index is reachable at all, far reaches it too. Conversely if far ever falls short of the current index, nothing could have reached that index, because far was already the best possible.

RULES THAT LOOK GREEDY BUT ARE NOT

Do not jump as far as possible each step, and do not jump to the cell with the biggest value. Both are *route-choosing* rules, and route choosing is exactly what this problem does not require. On $[2, 3, 1, 1, 4]$, jumping maximally from index 0 lands on index 2, whose value is 1 — a worse position than index 1.

The algorithm never decides where to jump. It only tracks how far jumping could get you.

8.5 The trace

Input $[3, 2, 1, 0, 4]$.

<i>i</i>	<i>a</i> [<i>i</i>]	<i>i</i> > <i>far</i> ?	<i>far</i> after
0	3	0 > 0 no	max(0, 3) = 3
1	2	1 > 3 no	max(3, 3) = 3
2	1	2 > 3 no	max(3, 3) = 3
3	0	3 > 3 no	max(3, 3) = 3
4	4	4 > 3 yes	return false

8.6 The code

Jump Game: reachability

```
bool canJump(vector<int>& a) {
    int far = 0;
    for (int i = 0; i < (int)a.size(); i++) {
        if (i > far) return false;      // stranded before reaching i
        far = max(far, i + a[i]);
    }
    return true;
}
```

$O(n)$ time, $O(1)$ space, one pass, no sorting. You can return early with `if (far >= n - 1) return true;` inside the loop, though it changes nothing asymptotically.

9 Jump Game II: the fewest jumps

Same array, same rules, harder question: what is the *minimum* number of jumps from index 0 to the last index? Assume it is reachable.

9.1 The realisation

Ask which indices you can be standing on after exactly k jumps. After 0 jumps: index 0. After 1 jump: everything in $1 \dots a[0]$. After 2 jumps: everything reachable from that band. The reachable set after k jumps is again a contiguous band, and the bands only move right.

So the answer to “fewest jumps” is “which band contains the last index”, and you find it by scanning once and counting how many times you cross a band boundary. This is breadth-first search on a graph, done without building the graph.

THE GREEDY CHOICE

Sweep left to right keeping three numbers: *far*, the farthest index the current band can reach; *end*, the last index of the current band; and *jumps*. Whenever i reaches *end*, a band has been fully explored, so increment *jumps* and set *end* = *far*.

9.2 The picture

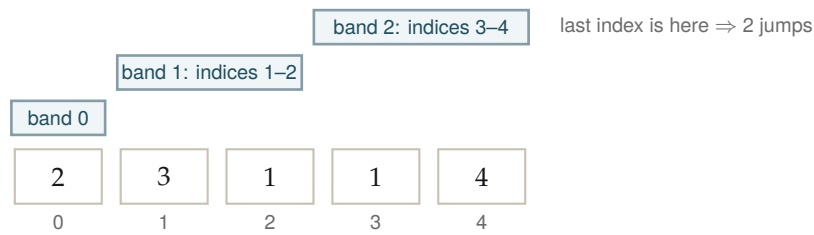


Figure 16. $[2, 3, 1, 1, 4]$ split into bands. Band k is the set of indices reachable in exactly k jumps; the answer is the band number containing $n - 1$.

9.3 The trace

Input $[2, 3, 1, 1, 4]$, $n = 5$, so the loop runs for $i = 0 \dots 3$.

i	far becomes	i == end?	jumps	end becomes
0	$\max(0, 0 + 2) = 2$	yes ($end = 0$)	1	2
1	$\max(2, 1 + 3) = 4$	no	1	2
2	$\max(4, 2 + 1) = 4$	yes ($end = 2$)	2	4
3	$\max(4, 3 + 1) = 4$	no	2	4

Answer: 2.

9.4 The code

Jump Game II: minimum jumps

```
int jump(vector<int>& a) {
    int jumps = 0, end = 0, far = 0;

    for (int i = 0; i + 1 < (int)a.size(); i++) { // note: stop before the last index
        far = max(far, i + a[i]);
        if (i == end) { // finished the current band
            jumps++;
            end = far;
        }
    }
    return jumps;
}
```

WHY THE LOOP STOPS ONE EARLY

If the loop included the last index, then arriving at $i = n - 1$ with $end == n - 1$ would trigger one more `jumps++` — counting a jump you never needed to make, because you are already at the destination. Running to $i < n - 1$ is not an optimisation; it is the difference between right and wrong. Test with $[0]$, which must return 0.

$O(n)$ time, $O(1)$ space.

10 Gas station

10.1 The problem

n stations sit on a circular route. At station i you can pick up $\text{gas}[i]$ litres, and driving from station i to station $i + 1$ burns $\text{cost}[i]$ litres. You start with an empty tank at a station of your choice and must complete the full loop without the tank ever going negative. Return a valid starting station, or -1 .

The tank has no capacity limit, which is what makes the problem tractable: the only thing that can go wrong is running out, never overflowing.

10.2 Reduce to one array

Station i is worth exactly

$$b_i = \text{gas}[i] - \text{cost}[i]$$

litres — positive if it gives you more than it takes, negative if it drains you. Two arrays become one, and the question becomes: *starting where, does every prefix sum of the circular sequence b stay non-negative?*

Station i	0	1	2	3	4
gas	1	2	3	4	5
cost	3	4	5	1	2
b_i	-2	-2	-2	+3	+3

10.3 The first check

FEASIBILITY

If $\sum_i b_i < 0$ the loop burns more fuel than the whole route supplies, so no start works. If $\sum_i b_i \geq 0$, a valid start always exists.

The first half is obvious. The second half is not, and it is the reason the algorithm can get away with returning a single candidate without verifying it. The proof falls out of the argument below.

10.4 The picture

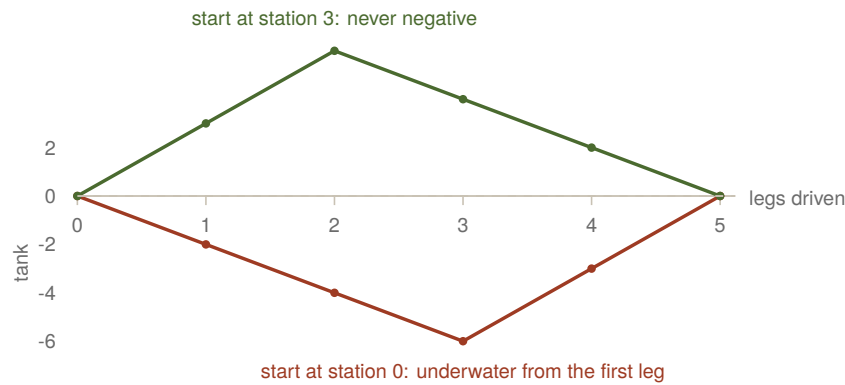


Figure 17. The same route, two starting stations. Both curves end at the same height — the total is fixed — but only one of them stays above the axis the whole way. The problem is entirely about the *shape* of the curve, not its endpoint.

10.5 The guess

Walk once from station 0, accumulating. The moment the running tank goes negative at station i , throw away the current candidate start and try $i + 1$.

THE GREEDY CHOICE

`cur += b[i]`. If `cur < 0`, set `start = i + 1` and `cur = 0`. Separately accumulate `total`. At the end, return `start` if `total >= 0`, else `-1`.

That skips a whole block of stations in one move, which is why the algorithm is linear rather than quadratic. Justifying the skip is the entire proof.

10.6 Why the whole block can be discarded

THE ELIMINATION ARGUMENT

Suppose you start at s and the tank first goes negative when arriving at station $i + 1$, that is

$$b_s + b_{s+1} + \dots + b_i < 0.$$

Take any station m strictly between s and i , and ask whether m could be a valid start.

Because the tank never went negative before $i + 1$, every prefix $b_s + \dots + b_{m-1}$ is ≥ 0 . Subtracting a non-negative quantity from a negative one leaves it negative:

$$\underbrace{b_m + \dots + b_i}_{\text{starting at } m} = \underbrace{(b_s + \dots + b_i)}_{<0} - \underbrace{(b_s + \dots + b_{m-1})}_{\geq 0} < 0.$$

So a driver starting at m also runs dry at or before station $i + 1$. Every candidate in $s \dots i$ dies the same way, and $i + 1$ is the next one worth testing.

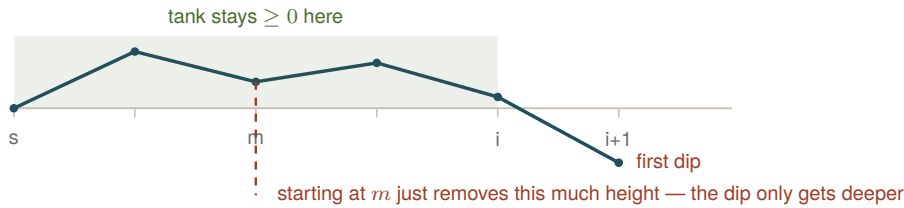


Figure 18. Restarting inside the block shifts the curve down, never up. Whatever made the journey fail is still there.

That also settles the feasibility claim. The loop ends with `start` set to some station after the last failure, and every station before it has been ruled out. If $\sum b_i \geq 0$, some start must work — and the only one not yet eliminated is `start`.

10.7 The trace

$b = [-2, -2, -2, +3, +3]$, `total` = 0.

i	b[i]	cur after	cur < 0?	start
0	-2	-2	yes	1 (cur reset to 0)
1	-2	-2	yes	2 (cur reset to 0)
2	-2	-2	yes	3 (cur reset to 0)
3	+3	3	no	3
4	+3	6	no	3

`total` = 0 ≥ 0 , so the answer is station 3.

10.8 The code

Gas station: find a valid start

```
int canCompleteCircuit(vector<int>& gas, vector<int>& cost) {
    int total = 0, cur = 0, start = 0;

    for (int i = 0; i < (int)gas.size(); i++) {
        int b = gas[i] - cost[i];
        total += b;
        cur += b;

        if (cur < 0) {           // everything from start..i is ruled out
            start = i + 1;
            cur = 0;
        }
    }
    return total ≥ 0 ? start : -1;
}
```

One pass, $O(n)$ time and $O(1)$ space. Note that the loop never wraps around — it does not need to, because `total` decides feasibility and the elimination argument decides the start.

CAREFUL

`total` and `cur` are different variables and both are needed. `cur` gets reset and therefore forgets the deficits; `total` never does, and it is the only thing that can tell you the answer is -1 . Merging them is the standard bug in this problem.

11

Choosing the rule: everything on one page

11.1 The nine problems

Problem	Greedy rule	Sort by	State carried
Interval merging	extend while the next start is inside the block	start, ascending	right end of the open block
Activity selection	take it if the hall is free	finish, ascending	last finish time
Minimum platforms	count arrivals and departures in time order	each array separately	current occupancy
Sweep line	process events left to right	coordinate	running total
Fractional knapsack	fill with the dearest material per kg	v/w , descending	remaining capacity
Job sequencing	richest job into the latest free slot	profit, descending	which slots are taken
Jump Game	track the farthest index reachable	nothing	far
Jump Game II	count band boundaries	nothing	far, end, jumps
Gas station	restart after the balance goes negative	nothing	cur, total, start

Two things are worth noticing in that table. First, the sort key is different almost every time — there is no “sort ascending and hope”. Second, the last three problems do not sort at all, and they are the linear ones; sorting is a cost you pay only when the rule needs an order.

11.2 Wrong rules and what kills them

Problem	Tempting but wrong	Counterexample	Correct rule
Activity selection	earliest start	one long activity covering the whole day beats nothing	earliest finish
Activity selection	shortest duration	a short activity straddling a boundary kills two long ones	earliest finish
Fractional knapsack	highest value	value 100 at weight 100 beaten by value 90 at weight 10	highest v/w
0/1 knapsack	highest v/w	$W=10$; item (12, 6) blocks two items (9, 5)	not greedy — use DP
Job sequencing	earliest deadline first	a worthless job with deadline 1 evicts a valuable one	highest profit first
Job sequencing	earliest free slot	wastes slot 1, which only deadline-1 jobs can use	latest free slot
Jump Game	always jump the maximum distance	[2, 3, 1, 1, 4] — the far landing has a smaller value	track far
Coin change	largest coin that fits	coins {1, 7, 10}, target 15	depends on the coin set; in general DP

Every counterexample in that column has at most five elements. That is the practical takeaway: if a greedy rule is wrong, a tiny input proves it, so build tiny inputs before you build the solution.

11.3 A checklist for a new problem

Is there a quantity being maximised or minimised?

If the problem only asks whether something is possible, you may still be able to phrase it as “maximise how far I get”.

Can I describe one choice that seems obviously safe?

Write it as a sentence with a comparison in it. If you cannot name the comparison, you do not have a rule yet.

Can I break it on three or four elements?

Try extremes: one huge element, one worthless one, exact ties, a zero.

Does exchanging my choice into an optimal solution keep it valid and no worse?

If yes, you have a proof. If you cannot argue it, treat the rule as unproven, not as true.

What is the one piece of state I carry across the loop?

Almost every greedy solution here carries one or two integers. If you need a table, you are probably writing DP.

broken $\Rightarrow$ back to the previous box

11.4 One line per problem

Worth memorising, but only after you can derive them.

Interval merging	sort the starts, stretch the right end
Activity selection	finish early, do more
Minimum platforms	count who is in the station right now
Sweep line	sort the events, watch the staircase
Fractional knapsack	best value per kilo first
Job sequencing	richest job, latest slot
Jump Game	remember only the farthest reach
Jump Game II	count the bands
Gas station	go negative, start again after the failure

12 Exercises

Work with pencil and paper first. Code only after you can state the rule and say why it is safe.

12.1 Warm-up: run the algorithms by hand

- E1. Merge $[1, 4]$ $[0, 2]$ $[3, 5]$ $[7, 9]$ $[8, 8]$. Which pairs actually overlapped, and how many intervals does the answer have?
- E2. Select the maximum number of activities from $(1, 4)$, $(3, 5)$, $(0, 6)$, $(5, 7)$, $(3, 9)$, $(5, 9)$, $(6, 10)$, $(8, 11)$. Write the sorted-by-finish order first.
- E3. Trains arrive at 200 210 300 320 350 and depart at 230 340 320 430 400. How many platforms? Draw the staircase.
- E4. Capacity 15, items $(v, w) = (10, 2)$, $(5, 3)$, $(15, 5)$, $(7, 7)$, $(6, 1)$, $(18, 4)$, $(3, 1)$. What is the maximum fractional value, and which item gets split?
- E5. Jobs with (deadline, profit): $(4, 20)$, $(1, 10)$, $(1, 40)$, $(1, 30)$. Which jobs run, and at which slots?
- E6. $[1, 2, 0, 1]$ — can you reach the end? $[2, 3, 0, 1, 4]$ — how many jumps at minimum?
- E7. $\text{gas} = [2, 3, 4]$, $\text{cost} = [3, 4, 3]$. Is a circuit possible, and from where?

12.2 Understanding: explain, do not code

- E8. In activity selection, why does the exchange argument need $f_G \leq f_X$ rather than $s_G \leq s_X$? Construct two activities where G starts earlier but finishes later, and show what goes wrong.
- E9. In job sequencing, suppose all profits are equal. Does the algorithm still do anything useful? What is it computing now?
- E10. Why does v/w order work for fractional knapsack but not for 0/1? Point at the exact sentence in the proof that stops being true.
- E11. In gas station, why is it safe to skip every station between the old start and the failure point, rather than just the old start?

13

Answers and hints

Full answers to the warm-ups, hints only for the rest.

13.1 Warm-up

- E1** Sorted: [0,2] [1,4] [3,5] [7,9] [8,8]. The first three chain together into [0,5]; [8,8] sits inside [7,9] and is swallowed, which is the case that needs the $\max$. Answer: [0,5] [7,9], two intervals.
- E2** By finish time: (1, 4), (3, 5), (0, 6), (5, 7), (3, 9), (5, 9), (6, 10), (8, 11). Take (1, 4), then (5, 7), then (8, 11). Answer: 3.
- E3** Sorted arrivals 200 210 300 320 350; sorted departures 230 320 340 400 430. The peak is 3, at time 320: the train that arrived at 210 is still in, the one that arrived at 300 is departing at exactly 320, and a third arrives at 320. Under the $a[i] \leq d[j]$ convention those three need three platforms.
- E4** Ratio order: (6, 1) pays 6, (10, 2) pays 5, (18, 4) pays 4.5, then (15, 5) and (3, 1) both pay 3, then (5, 3) pays 1.67 and (7, 7) pays 1. Filling in that order:
- | item | weight taken | running weight | running value |
|---------|--------------|----------------|------------------------------------|
| (6, 1) | 1 | 1 | 6 |
| (10, 2) | 2 | 3 | 16 |
| (18, 4) | 4 | 7 | 34 |
| (15, 5) | 5 | 12 | 49 |
| (3, 1) | 1 | 13 | 52 |
| (5, 3) | 2 of 3 | 15 | $52 + 5 \cdot \frac{2}{3} = 55.33$ |
- Total **55.33**, and the split item is (5, 3), taken two-thirds. The tie between (15, 5) and (3, 1) is harmless: both are taken whole, so their order does not matter.
- E5** Sorted by profit: (1, 40), (1, 30), (4, 20), (1, 10). The 40 takes slot 1. The 30 wants slot 1, which is gone, so it is rejected. The 20 takes slot 4. The 10 is rejected. Profit **60**, two jobs.
- E6** [1, 2, 0, 1]: far goes 1, 3, 3, 4 and never falls behind i , so **true**. [2, 3, 0, 1, 4]: bands are {0}, {1, 2}, {3, 4}, so **2 jumps**.
- E7** $b = [-1, -1, 1]$, total = $-1 < 0$, so -1 : no circuit exists, whatever the starting station.

13.2 Understanding

- E8** The activities after X in O are constrained by when the hall becomes *free*, which is f_X , not by when X began. Try $G = (0, 10)$ and $X = (1, 2)$: G starts earlier but occupies the hall until 10, so swapping it in destroys everything scheduled between 2 and 10.
- E9** With equal profits, “richest first” gives no information and the algorithm reduces to: place as many jobs as possible, each in its latest free slot. It is now computing the maximum number of jobs that can meet their deadlines — a scheduling feasibility count rather than a profit maximisation.

- E10** The sentence that fails is “swap one unit of material”. With indivisible items you cannot move one unit; you must move a whole item, which changes the weight and can leave capacity unusable. That leftover capacity is exactly what the counterexample in §6.5 exploits.
- E11** Because every prefix of the failed journey was non-negative. Starting later throws away a non-negative head, which lowers the curve rather than raising it — see the figure and algebra in §10.6.
- E12** Without sorting you cannot conclude anything when the next interval fails to touch the current block. On $[1,3]$ $[8,10]$ $[2,6]$ the scan opens $[1,3]$, sees $8 > 3$ and opens $[8,10]$, then sees $2 \leq 10$ and merges $[2,6]$ into the *wrong* block, leaving it unchanged. Output $[1,3]$ $[8,10]$; correct answer $[1,6]$ $[8,10]$.
- E13** Three places: the merge test becomes $x[0] \geq r$ instead of $x[0] > r$; activity selection already uses $s \geq \text{lastFinish}$, which is the touching-allowed convention, and would become $s > \text{lastFinish}$ under the other reading; minimum platforms flips $a[i] \leq d[j]$ to $a[i] < d[j]$. Nothing else in the workbook is affected.

13.3 Hints for the harder set

- E14** Maximising what you keep and minimising what you delete are the same problem. The one-line change is to subtract.
- E15** It does still work, but the proof needs care: “wherever it fits” must be made deterministic, and the exchange argument now swaps an activity between halls as well as into the schedule. Test it against inputs where one activity fits in both halls and one fits in neither.
- E16** The argument survives only if fuel is never wasted. With a cap, a station early in the block may fill the tank to C and lose the surplus, so the prefix you subtract is no longer guaranteed non-negative in the right sense. Look for a two-station example where restarting later genuinely helps.
- E17** It is not greedy. Landing on a cheap cell early can force expensive landings later, and no local rule sees that. This is shortest path on a DAG — either DP over indices or Dijkstra if tolls can be arbitrary.
- E18** Keep the free platforms in a priority queue keyed by the departure time of the train occupying them. Process trains in arrival order: reuse the platform that frees earliest if it is already free, otherwise open a new one. $O(n \log n)$, and it gives the assignment for free.

If a rule in these notes still feels like something you memorised rather than something you derived, go back to its WHY IT IS SAFE box and try to write the exchange argument from a blank page. That is the only part of greedy algorithms that transfers to a problem you have not seen.